École Supérieure Privée d'Ingénierie et de Technologies (ESPRIT)

Module : Architecture Web Distribuée

GeoRisk Habitat

Assurance habitation et risque d'inondation

Dossier de conception – Validation 1

Réalisé par : 4e Actuariat, groupe 6

Taha Yassine Snoussi	MS1 – property-service
Fathi Mejri	MS2 – policy-service
Mariem Amdouni	MS3 – pricing-service
Molka Baccari	MS4 – claims-service

Dépôt : `https://github.com/tahayassine-snoussi/GeoRisk-4Actuariat-g6`

Année universitaire 2026–2027

Table des matières

1 Description du sujet **2**
1.1 Contexte et problématique . . . 2
1.2 Solution proposée . . . 2
1.3 Périmètre de la première version . . . 2

2 Acteurs et fonctionnalités **2**
2.1 Acteurs . . . 2
2.2 Diagramme de cas d'utilisation . . . 3
2.3 Fonctionnalités principales . . . 3

3 Découpage en microservices **4**
3.1 Pourquoi une architecture microservices . . . 4
3.2 Principe du découpage . . . 4
3.3 Les services . . . 4

4 Données gérées par chaque service **5**
4.1 Une base par service . . . 5
4.2 MS1 – property-service . . . 6
4.3 MS2 – policy-service . . . 7
4.4 MS3 – pricing-service . . . 8
4.5 MS4 – claims-service . . . 9
4.6 MS5 – risk-engine-service . . . 10

5 Interactions entre les services **11**
5.1 Points d'accès REST . . . 11
5.2 Appels synchrones . . . 12
5.3 Événements asynchrones . . . 12
5.4 Exemples de scénarios . . . 14

6 Architecture globale **15**

7 Autres éléments de conception **15**
7.1 Sécurité . . . 15
7.2 Technologies . . . 16
7.3 Organisation de l'équipe . . . 16
7.4 Planning . . . 16
7.5 Risques identifiés . . . 16
7.6 Point à valider . . . 16

1 Description du sujet

1.1 Contexte et problématique

En assurance habitation, la prime est généralement fixée à partir de grandes catégories (type de logement, région). Le risque réel de la zone où se trouve le bien n'est pas vraiment mesuré, et l'assureur n'intervient qu'après le sinistre. Or, pour une inondation, quelques heures d'avance suffisent souvent à limiter les dégâts.

La question à laquelle nous voulons répondre est donc : comment tarifer plus justement le risque d'inondation d'un bien, prévenir l'assuré avant le sinistre, et se servir des sinistres réels pour améliorer les prévisions ?

1.2 Solution proposée

GeoRisk Habitat est une plateforme d'assurance habitation qui s'appuie sur un score de risque par zone géographique. Elle repose sur trois idées :

- *tarifer* : la prime dépend du score de risque de la zone (de 0 à 100) et des caractéristiques du bien ;
- *anticiper* : à partir des prévisions météo, on estime une probabilité d'inondation par zone et on alerte les assurés concernés ;
- *apprendre* : chaque alerte est comparée à ce qui s'est réellement passé (sinistre confirmé ou fausse alerte).

1.3 Périmètre de la première version

Nous traitons un seul aléa, l'inondation, sur dix zones prédéfinies (de Tunis Centre à Sfax). Les biens assurés sont des logements et des objets de valeur. Le parcours couvert va de la souscription jusqu'à l'indemnisation, qui reste simulée. L'incendie, le séisme ou la grêle ne sont pas traités, mais le modèle prévoit un champ `hazard` qui permettra de les ajouter.

2 Acteurs et fonctionnalités

2.1 Acteurs

Acteur	Rôle	Rôle Keycloak
Propriétaire	Demande une assurance pour un bien, accepte ou refuse la prime proposée, reçoit les alertes, déclare et suit ses sinistres	`OWNER`
Assureur	Gère les produits et les coefficients de tarification, affecte les experts, décide des indemnisations	`INSURER`
Expert	Évalue le dommage et dépose le rapport d'expertise	`EXPERT`
Administrateur	Gère le modèle de risque et lance les simulations de pluie	`ADMIN`
Compte technique	Utilisé par les services pour s'appeler entre eux	`SERVICE`
Open-Meteo	Système externe qui fournit les prévisions de pluie	–

Tab. 1 – Acteurs du système

2.2 Diagramme de cas d'utilisation

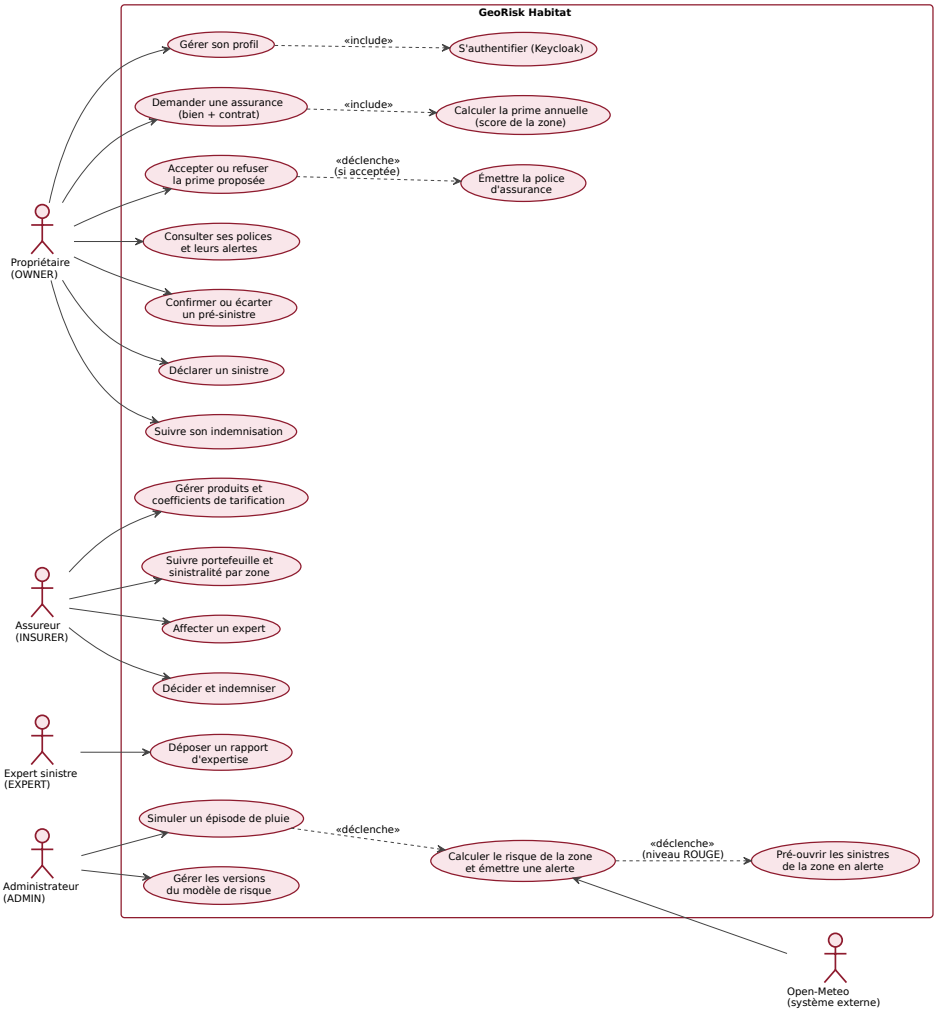


FIG. 1 – Diagramme de cas d'utilisation

2.3 Fonctionnalités principales

1. Le propriétaire demande une assurance pour un logement ou un objet ; le système retrouve la zone de risque du bien.
2. Le système calcule une prime annuelle et la propose au propriétaire, qui l'accepte ou la refuse. S'il accepte, une police est émise.
3. Lorsqu'une zone passe en alerte, les polices concernées sont signalées et, en cas d'alerte rouge, un pré-sinistre est ouvert pour chacune.
4. Le sinistre suit son cours : confirmation par l'assuré, expertise, décision de l'assureur et paiement simulé.
5. L'assureur ajuste les coefficients de tarification ; l'administrateur peut simuler un épisode de pluie pour la démonstration.

3 Découpage en microservices

3.1 Pourquoi une architecture microservices

Le choix des microservices se justifie ici par plusieurs besoins concrets :

- le moteur de risque est écrit en Python alors que le reste est en Spring Boot, ce qui n'est pas possible dans un monolithe ;
- lors d'une alerte, seuls les services des polices et des sinistres reçoivent un pic de charge, on peut donc les répliquer sans toucher au reste ;
- si le moteur de risque tombe, les devis continuent avec le dernier score connu ;
- chaque membre de l'équipe est responsable d'un service et d'un contexte métier, ce qui limite les conflits.

3.2 Principe du découpage

Nous avons découpé par métier de l'assureur et non par table : chaque service correspond à une activité réelle et répond à une seule question.

Service	Métier	Question traitée	Hors de son périmètre
MS1 property-service	Souscription	Qui est le client, que veut-il assurer, accepte-t-il le prix ?	Calcul de la prime, gestion des polices
MS2 policy-service	Gestion des polices	Qu'est-ce qui est couvert, et jusqu'à quand ?	Souscription, sinistres
MS3 pricing-service	Tarification	Combien ça coûte ?	Décision du client, évaluation du risque
MS4 claims-service	Sinistres	Que s'est-il passé, et combien rembourser ?	Définition des garanties
MS5 risk-engine-service	Analyse du risque	Quel est le risque de la zone, faut-il alerter ?	Prix et indemnités

Tab. 2 – Découpage par métier

La frontière entre MS1 et MS2 est la plus délicate : MS1 s'arrête au moment où le client accepte la prime, et MS2 crée la police à la réception de l'événement `contrat.accepte`. Il n'y a donc pas de dépendance synchrone dans les deux sens.

3.3 Les services

Service	Responsable	Technologie	Port	Rôle
MS1	T. Y. Snoussi	Spring Boot, H2	8081	Propriétaires, biens et contrats ; décision du client sur la prime
MS2	F. Mejri	Spring Boot, H2	8082	Polices, garanties et signalement des alertes
MS3	M. Amdouni	Spring Boot, H2	8083	Produits, coefficients et calcul de la prime
MS4	M. Baccari	Spring Boot, MySQL	8084	Sinistres, expertises et indemnisation
MS5	Équipe	FastAPI, PostgreSQL	8085	Zones, météo, score de risque et alertes

Tab. 3 – Les cinq microservices

MS5 est commun à l'équipe : chacun y développe un module (zones, ingestion météo, modèle, alertes et retour d'expérience).

4 Données gérées par chaque service

4.1 Une base par service

Chaque service est le seul propriétaire de ses données et aucune base n'est partagée. Un service ne lit jamais la base d'un autre : il passe par son API ou garde une copie locale mise à jour par événement (c'est le cas du cache des scores de zone dans MS3). Entre services, on ne conserve que des identifiants (`bienId`, `contratId`, `zoneId`), sans clé étrangère.

Service	Données	Base	Justification
MS1	Propriétaires, contrats, biens (logements et objets)	H2	Données simples et peu volumineuses, base embarquée
MS2	Polices, garanties, signalements d'alerte	H2	Modèle simple, contraintes d'unicité gérées par la base
MS3	Produits, coefficients, devis, cache des scores	H2	Données de référence peu volumineuses et souvent lues
MS4	Sinistres, expertises, paiements	MySQL	Montants financiers à conserver durablement, transactions ACID
MS5	Zones, météo, prédictions, alertes, résultats	PostgreSQL	Agrégations par zone, colonnes JSON, séries de pluie, bon support Python

Tab. 4 – Données et bases par service

4.2 MS1 – property-service

- `BienAssure` est une classe abstraite (héritage JPA `JOINED`) dont dérivent `Logement` et `Produit`.
- Il n'y a pas de lien direct entre `Proprietaire` et `BienAssure` : le bien est créé avec son contrat, et on retrouve le propriétaire à travers ses contrats.
- Un contrat est créé `EN_ATTENTE` avec la prime proposée par MS3, puis passe une seule fois à `ACCEPTE` ou `REFUSE`. L'acceptation n'est possible que si le devis est encore valide (30 jours).
- La valeur d'un logement doit être comprise entre 20 000 et 2 000 000 TND.

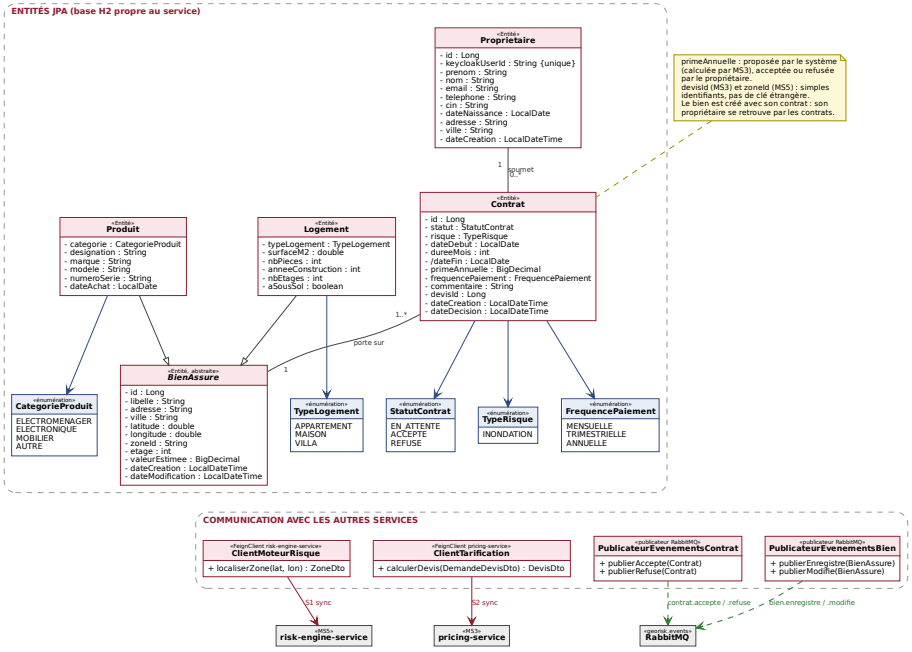


Fig. 2 – Diagramme de classes de MS1

4.3 MS2 – policy-service

- Une police est créée à la réception de `contrat.accepte` ; `contratId` est unique, donc un contrat ne peut produire qu'une seule police.
- Cycle de vie : `DRAFT` → `ACTIVE` → `EXPIRED` ou `CANCELLED`.
- Une alerte ne change pas le statut de la police : on lui ajoute un `PolicyAlertFlag`, unique pour chaque couple (alerte, police).

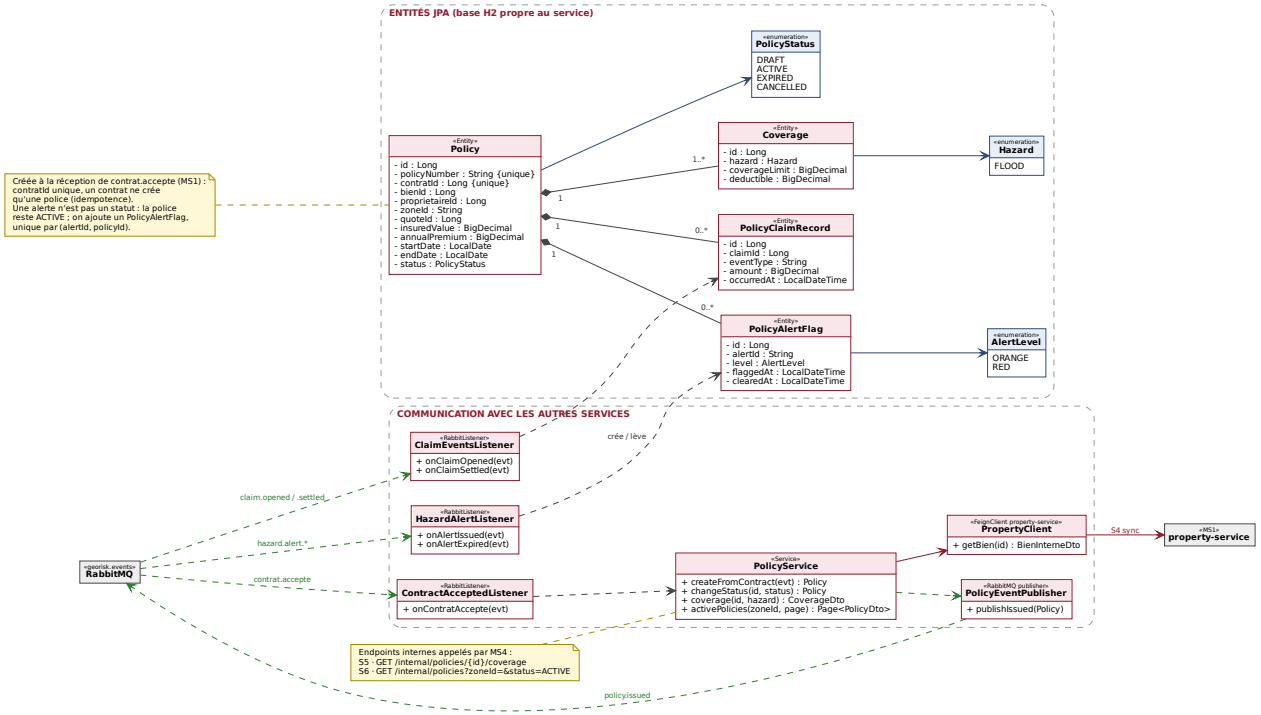


Fig. 3 – Diagramme de classes de MS2

4.4 MS3 – pricing-service

La prime pure est calculée ainsi :

$$\text{Prime pure} = V \times t \times C_{\text{zone}} \times C_{\text{type}} \times C_{\text{étage}} \times C_{\text{âge}}, \qquad C_{\text{zone}} = 0{,}8 + 1{,}7 \times \frac{\text{score}}{100}$$

où V est la valeur assurée et t le taux de base du produit. Pour un objet, C_{type} est remplacé par un coefficient de catégorie. On ajoute ensuite les frais, la marge et la taxe pour obtenir la prime annuelle.

Par exemple, pour une maison de 300 000 TND au rez-de-chaussée, construite en 2005, dans une zone de score 80 et avec un taux de 1 ‰, on obtient $300 \times 2{,}16 \times 1 \times 1{,}2 \times 1 = 777{,}60$ TND de prime pure.

Un devis est valide 30 jours, puis passe à `ACCEPTED` ou `REFUSED` selon la décision du client, ou à `EXPIRED`.

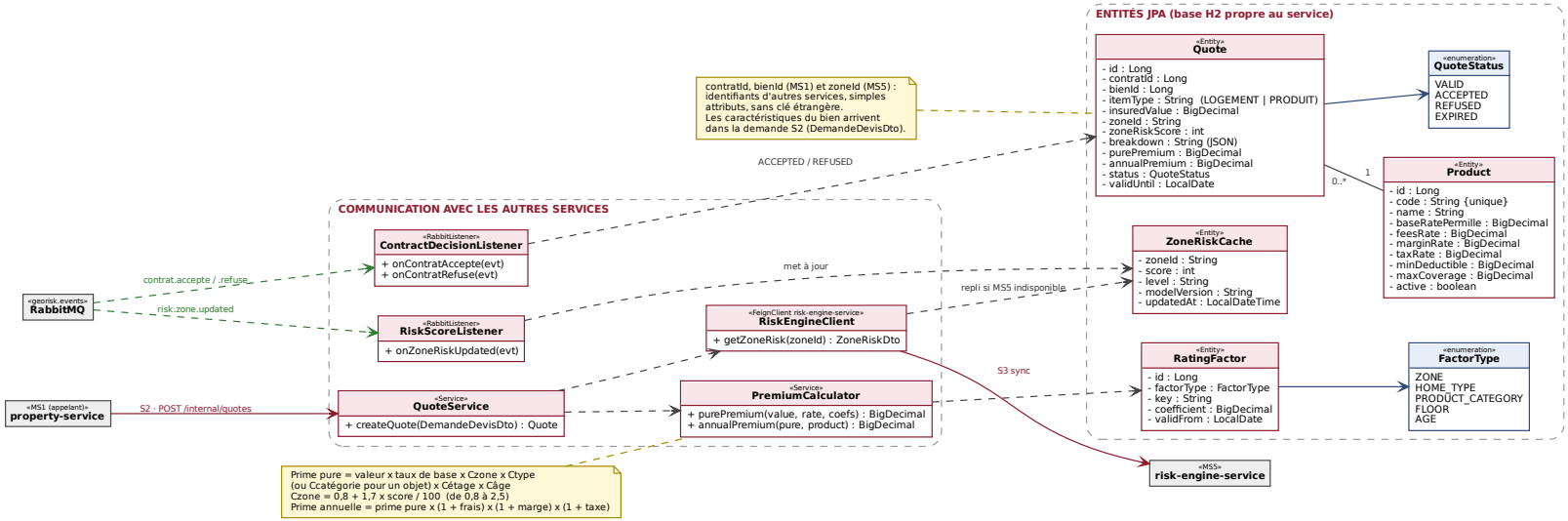


Fig. 4 – Diagramme de classes de MS3

4.5 MS4 – claims-service

- Cycle de vie : `PRE_OPENED` → `DECLARED` (ou `DISMISSED`) → `UNDER_EXPERTISE` → `ACCEPTED` ou `REJECTED` → `PAID` → `CLOSED`. Si la garantie n'a pas pu être vérifiée, la déclaration passe par `PENDING_CHECK`.
- Indemnité = $\min\big(\max(\text{dommage} - \text{franchise}, 0),\ \text{plafond}\big)$, le plafond et la franchise étant recopiés depuis MS2.
- Un seul pré-sinistre par couple (alerte, police).

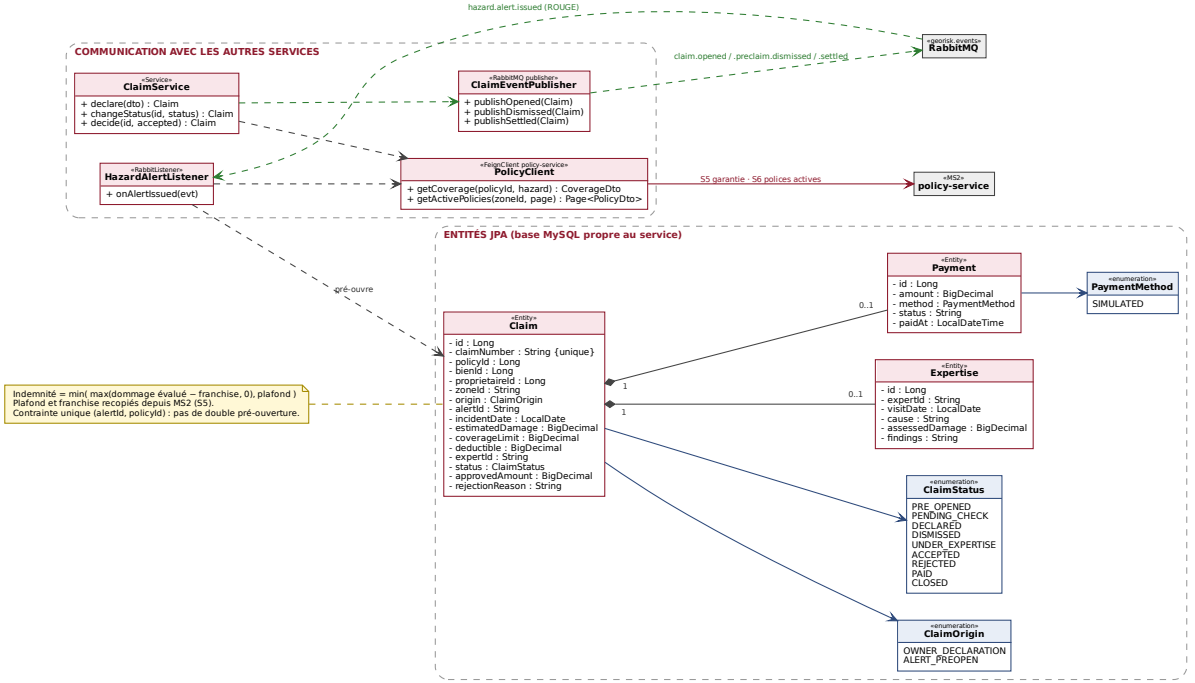


Fig. 5 – Diagramme de classes de MS4

4.6 MS5 – risk-engine-service

MS5 manipule deux notions différentes :

- le *score de zone* (0 à 100), qui représente le risque de fond de la zone et sert à la tarification ;
- la *probabilité d'inondation à 72 heures*, recalculée chaque heure à partir de la météo, qui sert uniquement aux alertes.

Les niveaux sont VERT, JAUNE, ORANGE et ROUGE. Une alerte n'est émise que si la zone reste au moins ORANGE sur deux évaluations consécutives, pour éviter de réagir à un pic isolé.

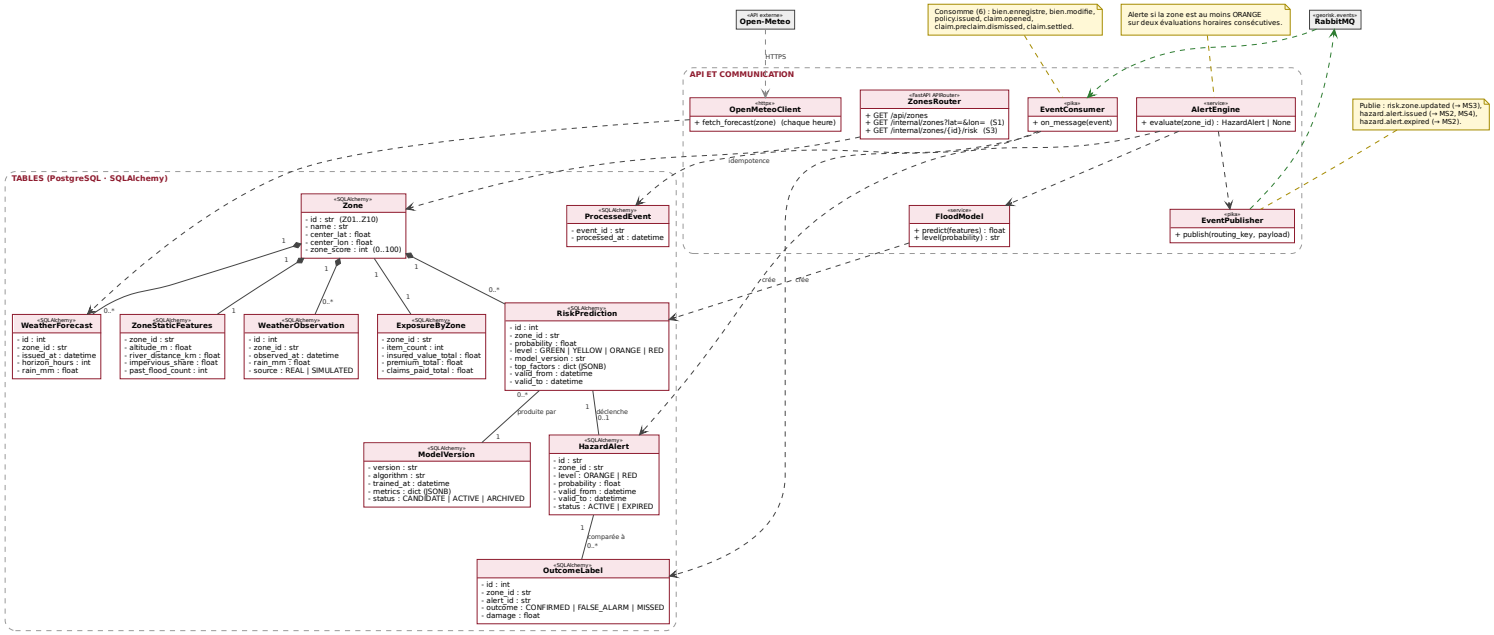


Fig. 6 – Modèle de données de MS5

5 Interactions entre les services

Les services communiquent de deux façons : en synchrone (OpenFeign et Eureka) lorsque l'appelant a besoin de la réponse pour continuer, et en asynchrone (RabbitMQ) pour annoncer qu'un fait s'est produit. Chaque service utilise les deux modes.

5.1 Points d'accès REST

Les routes `/api/**` sont exposées par l'API Gateway. Les routes `/internal/**` ne servent qu'aux appels entre services et ne sont jamais exposées. Le tableau suivant reprend les principales routes de chaque service.

Méthode	Route	Codes de retour
MS1 – property-service		
POST	`/api/contrats`	201, 400, 404
PATCH	`/api/contrats/{id}` (accepter ou refuser)	200, 400, 403, 404, 409
GET	`/api/contrats/{id}`	200, 403, 404
GET	`/internal/biens/{id}`	200, 404
MS2 – policy-service		
PATCH	`/api/policies/{id}` (activer, résilier)	200, 403, 404, 409
GET	`/api/policies/me/alerts`	200
GET	`/internal/policies/{id}/coverage`	200, 404
GET	`/internal/policies?zoneId=&status=ACTIVE`	200
MS3 – pricing-service		
GET	`/api/quotes/{id}`	200, 404
PUT	`/api/rating-factors/{id}`	200, 400, 404
POST	`/internal/quotes`	201, 400, 404
MS4 – claims-service		
POST	`/api/claims`	201, 400, 409
PATCH	`/api/claims/{id}`	200, 403, 404, 409
POST	`/api/claims/{id}/expertise`	201, 400, 403, 409
POST	`/api/claims/{id}/decision`	201, 400, 409
MS5 – risk-engine-service		
GET	`/api/zones/{id}/risk`	200, 404
POST	`/api/weather-simulations`	201, 400, 404
GET	`/internal/zones?lat=&lon=`	200, 400
GET	`/internal/zones/{id}/risk`	200, 404

Tab. 5 – Principales routes REST

Les erreurs sont renvoyées au format Problem Details (RFC 9457). Les codes 401 (jeton absent ou expiré) et 403 (droits insuffisants) s'appliquent à toutes les routes protégées.

5.2 Appels synchrones

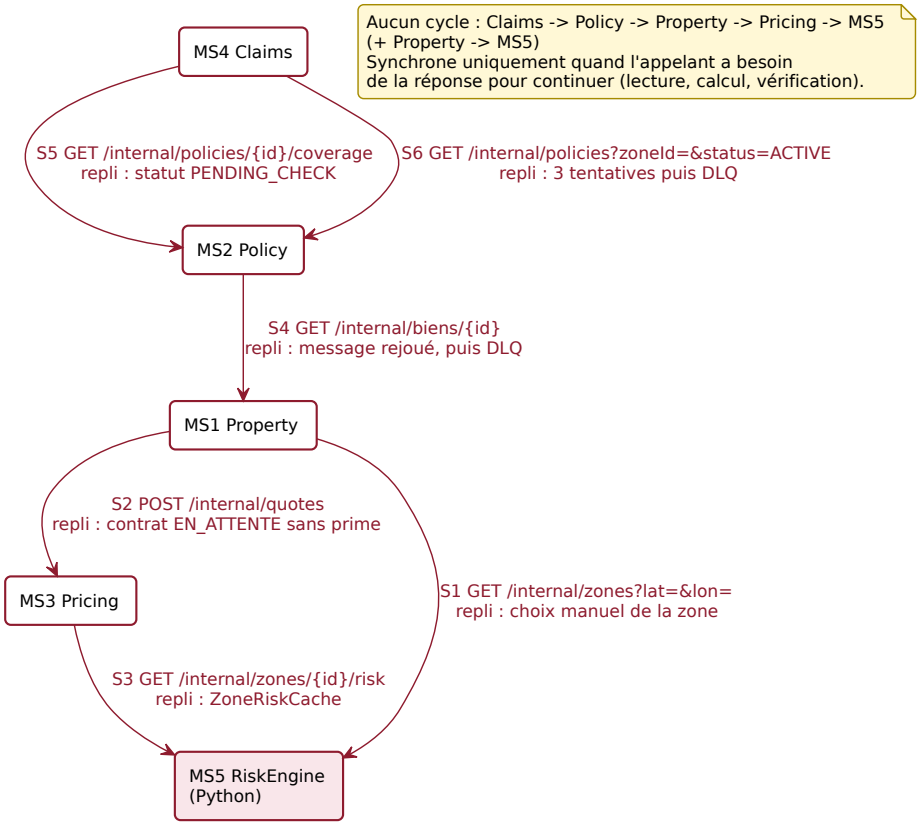


FIG. 7 – Appels synchrones (la flèche part de l'appelant)

	Appel	Route	En cas de panne
S1	MS1 → MS5	`GET /internal/zones?lat=&lon=`	Le propriétaire choisit la zone dans une liste
S2	MS1 → MS3	`POST /internal/quotes`	Contrat créé sans prime, redemandée plus tard
S3	MS3 → MS5	`GET /internal/zones/{id}/risk`	Dernier score connu (cache local)
S4	MS2 → MS1	`GET /internal/biens/{id}`	Message rejoué puis mis en file d'erreurs
S5	MS4 → MS2	`GET /internal/policies/{id}/coverage`	Sinistre mis en `PENDING_CHECK`
S6	MS4 → MS2	`GET /internal/policies?zoneId=&status=ACTIVE`	Message rejoué puis mis en file d'erreurs

TAB. 6 – Les six appels synchrones

Chaque client Feign a un délai maximal et un disjoncteur Resilience4j. Les appels ne forment aucun cycle : MS4 → MS2 → MS1 → MS3 → MS5, plus MS1 → MS5.

5.3 Événements asynchrones

Tous les événements passent par un seul exchange RabbitMQ de type *topic*, `georisk.events`. Chaque consommateur a sa propre file, avec une file d'erreurs. Le message n'est acquitté qu'après

traitement, et un message reçu deux fois ne produit qu'un seul effet.

Événement	Producteur	Consommateurs	Effet
`bien.enregistre`, `bien.modifie`	MS1	MS5	Met à jour l'exposition de la zone
`contrat.accepte`	MS1	MS2, MS3	Crée la police ; devis accepté
`contrat.refuse`	MS1	MS3	Devis refusé
`policy.issued`	MS2	MS5	Exposition et primes par zone
`risk.zone.updated`	MS5	MS3	Met à jour le cache des scores
`hazard.alert.issued`	MS5	MS2, MS4	Polices signalées ; pré-sinistres si ROUGE
`hazard.alert.expired`	MS5	MS2	Lève le signalement
`claim.opened`	MS4	MS2, MS5	Sinistre confirmé
`claim.preclaim.dismissed`	MS4	MS5	Fausse alerte
`claim.settled`	MS4	MS2, MS5	Montant réglé, recalcul du score

Tab. 7 – Les onze événements

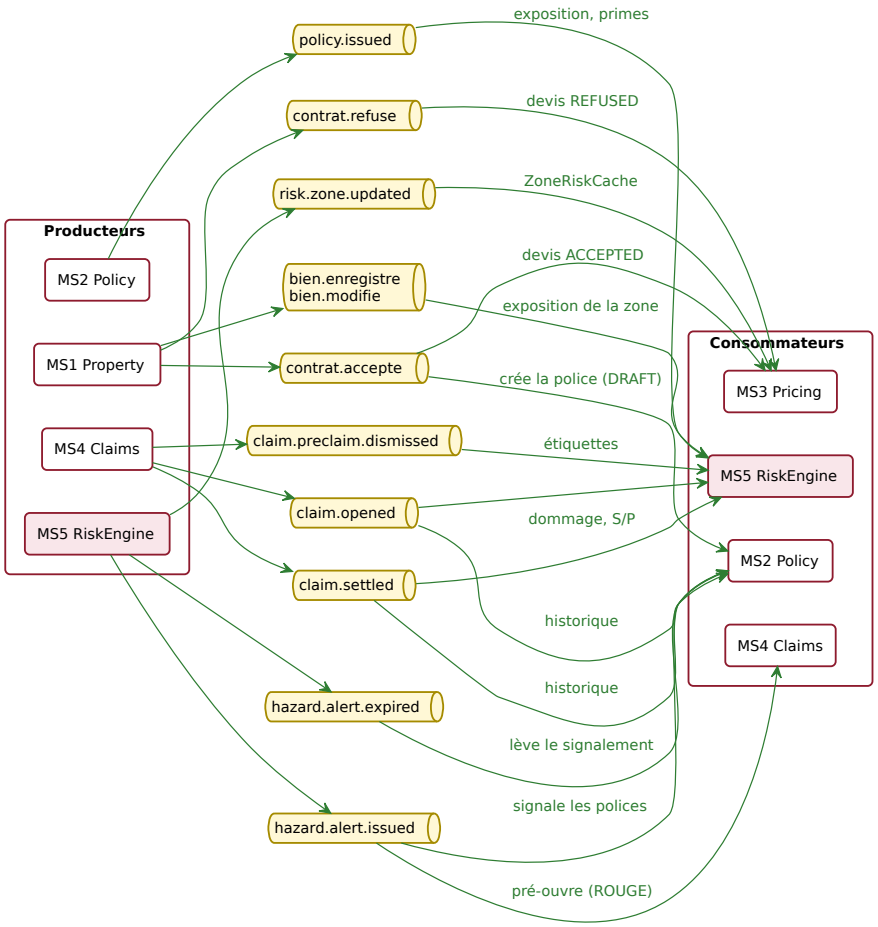


Fig. 8 – Producteurs, événements et consommateurs

5.4 Exemples de scénarios

La souscription montre l'enchaînement complet : MS1 demande la prime à MS3, qui lit le score de la zone chez MS5. Le propriétaire accepte, MS1 publie `contrat.accepte`, et MS2 crée la police.

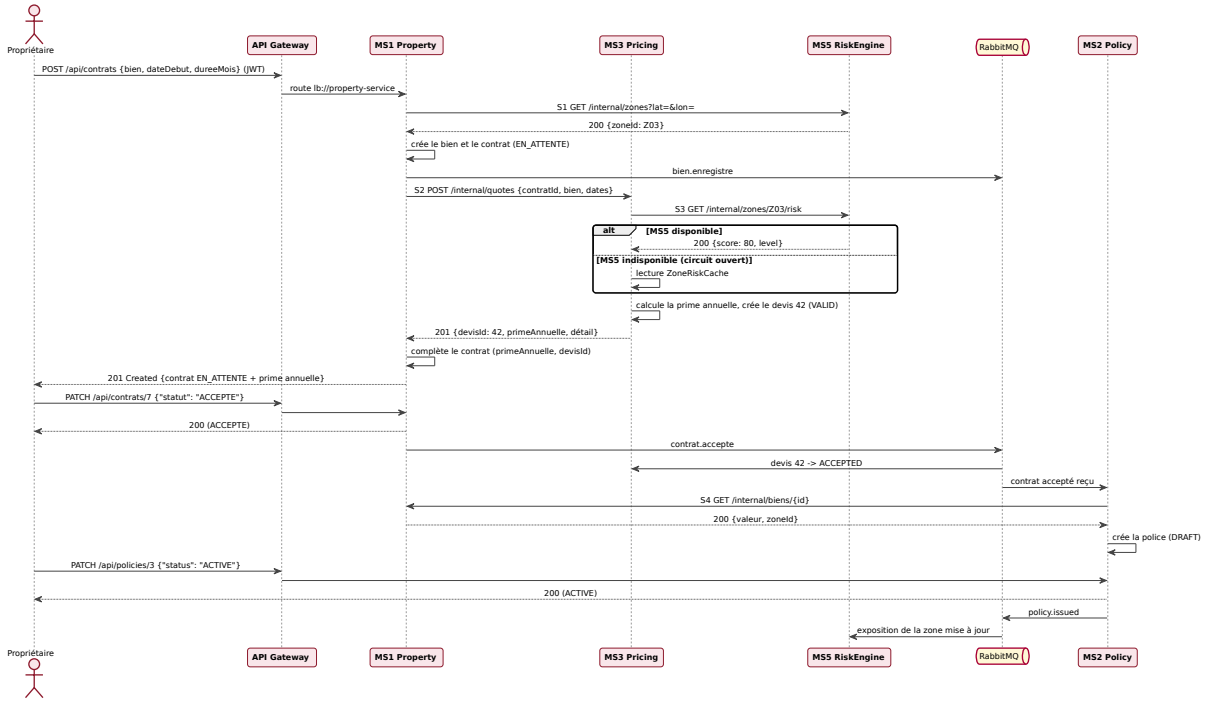


Fig. 9 – Séquence de souscription

Lors d'une alerte, MS5 publie `hazard.alert.issued` : MS2 signale les polices de la zone et, si l'alerte est rouge, MS4 ouvre un pré-sinistre pour chacune. Le propriétaire confirme ou écarte le pré-sinistre, ce qui permet à MS5 de mesurer la qualité de ses alertes.

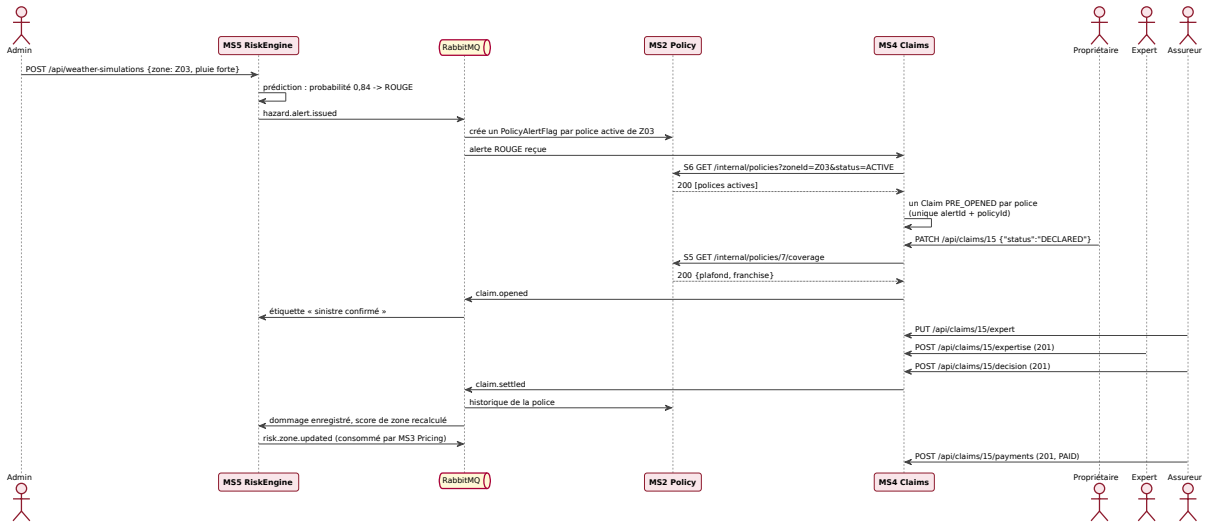


Fig. 10 – Séquence d'une alerte inondation

6 Architecture globale

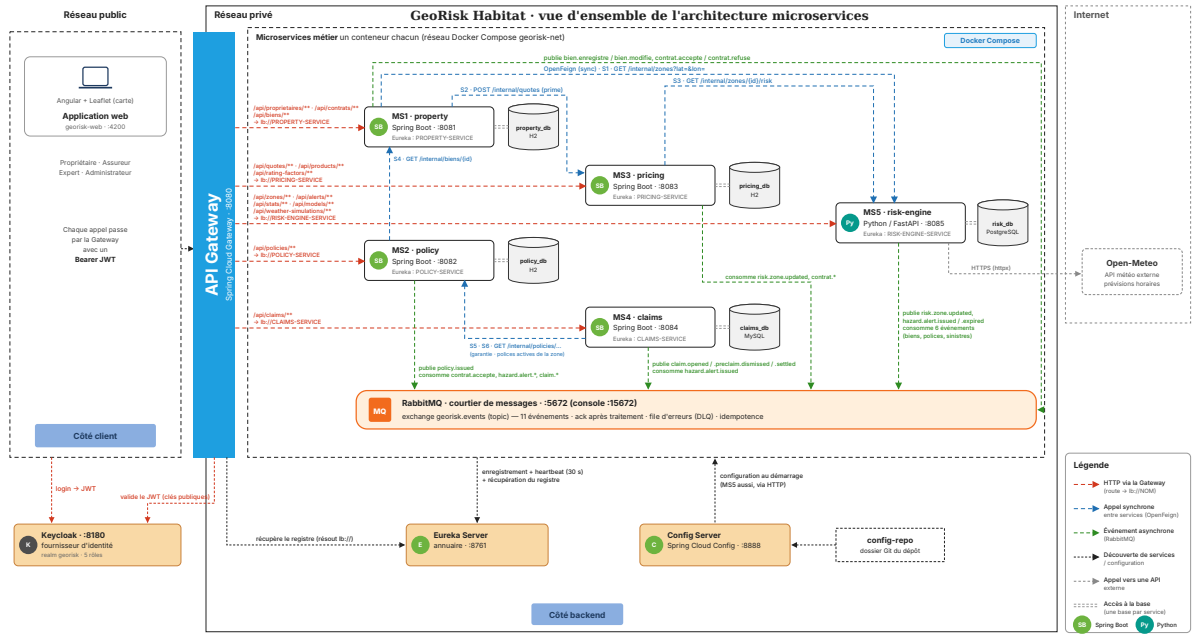


FIG. 11 – Architecture globale

Le client passe toujours par l'API Gateway, qui vérifie le jeton JWT et redirige la requête vers le bon service grâce à Eureka. Les autres composants d'infrastructure sont les suivants :

Composant	Technologie	Port	Rôle
API Gateway	Spring Cloud Gateway	8080	Point d'entrée unique, routage, vérification du JWT
Eureka	Spring Cloud Netflix	8761	Découverte des services
Config Server	Spring Cloud Config	8888	Configuration centralisée
Keycloak	Keycloak	8180	Comptes, rôles et jetons
RabbitMQ	RabbitMQ	5672	Transport des événements
Frontend	Angular, Leaflet	4200	Interface et carte des zones

TAB. 8 – Composants d'infrastructure

L'ensemble est lancé avec Docker Compose, dans l'ordre : Config Server, Eureka, puis Keycloak, RabbitMQ et les bases, puis les microservices, et enfin la Gateway et le frontend.

7 Autres éléments de conception

7.1 Sécurité

La sécurité repose sur Keycloak (realm `georisk`, cinq rôles). Le frontend obtient un jeton JWT, envoyé à chaque requête ; aucune session n'est gardée côté serveur. La Gateway vérifie le jeton, puis chaque service le vérifie à nouveau. Un propriétaire ne peut accéder qu'à ses propres contrats, polices et sinistres (sinon 403). Pour les appels entre services, chaque service utilise un compte technique avec le rôle `SERVICE`.

7.2 Technologies

- Services métier : Java 17, Spring Boot 3, Spring Data JPA.
- Infrastructure : Spring Cloud Gateway, Eureka, Config Server, OpenFeign, Resilience4j, RabbitMQ, Keycloak.
- MS5 : Python, FastAPI, SQLAlchemy, PostgreSQL.
- Frontend : Angular avec Leaflet pour la carte.
- Déploiement : Docker Compose. Suivi : GitHub et GitHub Projects.

7.3 Organisation de l'équipe

Étudiant	Service	Partie d'infrastructure	Module MS5
Taha Yassine Snoussi	MS1	API Gateway, Eureka	zones
Fathi Mejri	MS2	Config Server, OpenFeign	ingestion météo
Mariem Amdouni	MS3	RabbitMQ	modèle
Molka Baccari	MS4	Keycloak	alertes

Tab. 9 – Répartition des tâches

Le dépôt contient les dossiers `Backend` (un projet par service), `Frontend` et `Documentation`. Chacun travaille sur la branche de son service et fusionne dans `master` par pull request.

7.4 Planning

Séances	Travail prévu
S4	Validation 1 : conception et contrats d'API
S5 – S6	Gateway, Eureka, Config Server, appels OpenFeign, version simplifiée de MS5
S7 – S8	RabbitMQ, Keycloak, alertes de MS5
S9 – S10	Docker Compose, frontend, tests, puis validation 2

Tab. 10 – Planning prévisionnel

7.5 Risques identifiés

- MS5 en Python non accepté : nous le referions en Spring Boot avec le même contrat d'API, sans impact sur les autres services.
- Retard d'un service : les contrats d'API sont figés dès cette validation, et une version simplifiée de MS5 avec des scores fixes sera disponible dès la séance 5.
- Météo calme le jour de la démonstration : une route permet de simuler un épisode de pluie.

7.6 Point à valider

La consigne impose MongoDB ou PostgreSQL pour le cinquième microservice, ce que nous respectons avec PostgreSQL. En revanche, nous souhaitons confirmer que l'utilisation de Python (FastAPI) est acceptée pour ce service.